Here is the **real-time (practical) rule-set** used in industry for choosing between **String vs StringBuilder vs StringBuffer** — with examples.

✅ Real-Time Decision Table (Industry Rule)

Situation in Software	Use	Why
Fixed text / constants (UI labels, messages, config keys)	✅ String	No modification needed, safest & clean
String concatenation in loop / repeated building	✅ StringBuilder	Fastest, avoids new object creation
Multiple threads modifying SAME string object	✅ StringBuffer	Thread-safe (synchronized), prevents race conditions
Logging statements / exception messages	✅ StringBuilder	fast string building
SQL query building dynamically	✅ StringBuilder	faster and cleaner query generation
JSON/XML building in code	✅ StringBuilder	best performance
Web apps (Spring Boot) building response message	✅ StringBuilder	single-thread request handling mostly
Multi-threaded shared buffer (rare)	✅ StringBuffer	safe in concurrent modifications
When using + operator in normal small cases	✅ String (or "+")	compiler uses StringBuilder internally

---

✅ Real-Time Thumb Rules (Easy to Remember)

✅ Rule 1: If no changes → use String

```
String country = "India";
String msg = "Welcome to STS!";
```

✅ Rule 2: If you are MODIFYING repeatedly → use StringBuilder

Especially inside loops.

❌ Bad

```
String s = "";
for(int i=0; i<10000; i++){
   s = s + i;   // very slow, creates many objects
}
```

✅ Good

```
StringBuilder sb = new StringBuilder();
for(int i=0; i<10000; i++){
   sb.append(i);
}
String result = sb.toString();
```

✅ Rule 3: If multiple threads modify same text → use StringBuffer

Example: shared global buffer between threads (rare in modern apps)

```
StringBuffer buffer = new StringBuffer();

Thread t1 = new Thread(() -> buffer.append("A"));
Thread t2 = new Thread(() -> buffer.append("B"));

t1.start();
t2.start();
```

✅ Real-Time Usage Examples (Software)

✅ 1) Spring Boot / REST API message creation

✅ Best: StringBuilder

```
public String createResponse(String name, int id){
   StringBuilder sb = new StringBuilder();
   sb.append("User created: ");
   sb.append(name);
   sb.append(" with id=");
   sb.append(id);
   return sb.toString();
}
```

➡ Because each request is handled separately (mostly), shared-thread modification rarely happens.

✅ 2) Logging

✅ Best: StringBuilder or logger placeholders

```
StringBuilder sb = new StringBuilder();
sb.append("OrderId=").append(orderId)
  .append(", Amount=").append(amount);

System.out.println(sb.toString());
```

✅ 3) SQL Query Building

✅ Best: StringBuilder

```
StringBuilder query = new StringBuilder();
query.append("SELECT * FROM users WHERE 1=1 ");

if(name != null){
   query.append("AND name='").append(name).append("' ");
}
if(city != null){
   query.append("AND city='").append(city).append("' ");
}

System.out.println(query);
```

✅ 4) Multithread shared data buffer (Server / Background Jobs)

✅ Best: StringBuffer (if shared object is required)

```
StringBuffer shared = new StringBuffer();

Runnable task = () -> {
   for(int i=0; i<1000; i++){
      shared.append(i);
   }
};

new Thread(task).start();
new Thread(task).start();
```

✅ Performance Reality in Industry

Class	Speed	Real-world note
String	slow if modified repeatedly	best for fixed values
StringBuffer	slower than builder	synchronization overhead
StringBuilder	✅ fastest	most used in real apps

✅ What Real Companies Follow (Practical Standard)

✅ **90% real-world code uses:**

- String for constants and normal text
- StringBuilder for concatenations in loops / building output

StringBuffer is **rare in modern applications**, because:

- synchronization overhead
- better alternatives exist: StringBuilder + external locking, or thread-local builder

---

✅ Final Shortcut (1 Line)

✅ **String** = read-only
✅ **StringBuilder** = fast editing (single-thread)
✅ **StringBuffer** = safe editing (multi-thread)

If you want, I can also give a **benchmark program** (String vs StringBuilder vs StringBuffer timing) that you can run in **STS console** and see real performance difference.